

Phase 3 Test Specifications

POC-01 — Loan Application Management System

Total Test Cases: 20 | **Pass Threshold:** 14 of 20 (70%)

TOOL DEFINITION TESTS (4 cases)

TC-01-P3-TOOL-01: Tool Schema Valid

Category: Tool Definition | **Priority:** High

Steps:

1. Import all tools from `agent/tools.py`
2. Verify each tool has a `name` and `description` attribute

Pass Criteria: All 5 tools have non-empty `name` and `description` attributes

```
def test_tool_schema_valid():
    from agent.tools import (get_application_details, list_applications,
                             get_dashboard_summary, search_loan_policy,
                             get_applicant_details)

    tools = [get_application_details, list_applications, get_dashboard_summary,
              search_loan_policy, get_applicant_details]

    for tool in tools:
        assert hasattr(tool, 'name'), f"Tool missing name"
        assert hasattr(tool, 'description'), f"Tool {tool.name} missing
description"
        assert len(tool.name) > 0
        assert len(tool.description) > 10 # Must have meaningful description
```

TC-01-P3-TOOL-02: All Tools Registered in Agent

Category: Tool Definition | **Priority:** High

Steps:

1. Build the agent executor
2. Verify all 5 tools are accessible

Pass Criteria: `executor.tools` contains all 5 tool names

```
def test_tools_registered_in_agent():
    from agent.agent import build_agent
    executor = build_agent()

    tool_names = {t.name for t in executor.tools}
    expected = {"get_application_details", "list_applications",
                "get_dashboard_summary",
                "search_loan_policy", "get_applicant_details"}

    assert expected.issubset(tool_names), f"Missing tools: {expected - tool_names}"
```

TC-01-P3-TOOL-03: Tool Descriptions Are Specific and Actionable

Category: Tool Definition | **Priority:** Medium

Steps:

1. Read description of `get_application_details`
2. Verify it contains "Use this when" or equivalent guidance

Pass Criteria: Description contains usage guidance; minimum 50 characters

```
def test_tool_description_present():
    from agent.tools import get_application_details

    desc = get_application_details.description
    assert len(desc) >= 50, f"Description too short: {desc}"
    # Should indicate when to use it
    usage_indicators = ["use this", "when the user", "use when", "for questions"]
    assert any(ind in desc.lower() for ind in usage_indicators)
```

TC-01-P3-TOOL-04: Tool Input Validation

Category: Tool Definition | **Priority:** Medium

Steps:

1. Call `get_application_details` with a valid string input directly
2. Verify no type error is raised

Pass Criteria: Tool accepts string input without TypeError

```
def test_tool_input_validation():
    from agent.tools import get_application_details
    # Direct tool call (bypasses LLM)
```

```
try:
    result = get_application_details.invoke("1")
    assert isinstance(result, str)
except TypeError as e:
    pytest.fail(f"Tool rejected valid string input: {e}")
```

TOOL EXECUTION TESTS (6 cases)

Fixture:

```
@pytest.fixture
def running_api():
    """Assumes Phase 1 API is running on http://localhost:8000"""
    import requests
    try:
        resp = requests.get("http://localhost:8000/api/v1/dashboard/summary",
                             headers={"Authorization": f"Bearer {os.getenv('API_JWT_TOKEN')}"})
        if resp.status_code not in [200, 401]:
            pytest.skip("Phase 1 API not running")
    except:
        pytest.skip("Phase 1 API not running")
```

TC-01-P3-EXEC-01: get_application_details Tool Returns Data

Category: Tool Execution | **Priority:** High **Prerequisites:** Phase 1 API running, at least 1 application in database

```
def test_get_application_tool_returns_data(running_api):
    from agent.tools import get_application_details
    result = get_application_details.invoke("1")

    assert isinstance(result, str)
    assert "Application ID" in result or "not found" in result.lower()
    # If application 1 exists:
    if "Application ID" in result:
        assert "Loan Type" in result
        assert "Status" in result
```

TC-01-P3-EXEC-02: list_applications Tool Filters Correctly

Category: Tool Execution | **Priority:** High **Prerequisites:** Phase 1 API running

```
def test_list_applications_tool_filters_correctly(running_api):
    from agent.tools import list_applications

    result = list_applications.invoke({"status": "submitted", "loan_type": ""})
    assert isinstance(result, str)
    # Either found applications or "no applications found"
    assert len(result) > 0
```

TC-01-P3-EXEC-03: get_dashboard_summary Tool Returns Summary

Category: Tool Execution | **Priority:** High **Prerequisites:** Phase 1 API running

```
def test_dashboard_tool_returns_summary(running_api):
    from agent.tools import get_dashboard_summary
    result = get_dashboard_summary.invoke({})

    assert "Dashboard Summary" in result or "Total Applications" in result
    assert isinstance(result, str)
```

TC-01-P3-EXEC-04: search_loan_policy Tool Returns Answer

Category: Tool Execution | **Priority:** High **Prerequisites:** ChromaDB populated from Phase 2

```
def test_search_policy_tool_returns_answer():
    from agent.tools import search_loan_policy
    result = search_loan_policy.invoke("What documents are required for a personal loan?")

    assert isinstance(result, str)
    assert len(result) > 20
    assert any(doc in result.lower() for doc in ["id_proof", "income_proof", "bank_statement", "document"])
```

TC-01-P3-EXEC-05: Tool Error Handled When API Returns 404

Category: Tool Execution | **Priority:** High

```
def test_get_application_tool_handles_404(running_api):
    from agent.tools import get_application_details
    result = get_application_details.invoke("99999")

    assert isinstance(result, str)
```

```
assert "not found" in result.lower() or "error" in result.lower()
# Must NOT raise an exception
```

TC-01-P3-EXEC-06: Tool Handles API Unavailable

Category: Tool Execution | **Priority:** Medium **Note:** Temporarily point tool to wrong port to simulate unavailability

```
def test_api_unavailable_handled(monkeypatch):
    from agent import tools
    monkeypatch.setenv("API_BASE_URL", "http://localhost:9999/api/v1")

    # Re-import to pick up new env var
    import importlib
    importlib.reload(tools)

    result = tools.get_application_details.invoke("1")
    assert isinstance(result, str)
    assert "unavailable" in result.lower() or "error" in result.lower() or
    "connection" in result.lower()
```

CONTEXT MANAGEMENT TESTS (4 cases)

TC-01-P3-CTX-01: System Prompt Contains Role Definition

Category: Context Management | **Priority:** High

```
def test_system_prompt_contains_context():
    from agent.prompts import LOAN_AGENT_SYSTEM_PROMPT

    assert "loan" in LOAN_AGENT_SYSTEM_PROMPT.lower()
    assert "assistant" in LOAN_AGENT_SYSTEM_PROMPT.lower()
    assert len(LOAN_AGENT_SYSTEM_PROMPT) > 200
    # Should contain tool selection guidance
    assert "when" in LOAN_AGENT_SYSTEM_PROMPT.lower()
```

TC-01-P3-CTX-02: Long Response Summarized

Category: Context Management | **Priority:** High

```
def test_long_response_summarized():
    from agent.summarizer import summarize_text

    long_text = "Application data: " + ("ID: 1, Status: submitted, Amount: 100000.
```

```
" * 100)
    assert len(long_text) > 2000

    summary = summarize_text(long_text)

    assert isinstance(summary, str)
    assert len(summary) < len(long_text), "Summary should be shorter than
original"
    assert len(summary) > 0
```

TC-01-P3-CTX-03: Short Response Not Summarized

Category: Context Management | **Priority:** Medium

```
def test_short_response_not_summarized():
    from agent.tools import _summarize_if_long

    short_text = "Application 1: Personal loan, ₹100,000, Status: Approved"
    assert len(short_text) < 2000

    result = _summarize_if_long(short_text)
    assert result == short_text # Should be returned unchanged
```

TC-01-P3-CTX-04: Prompt Template Renders Correctly

Category: Context Management | **Priority:** Medium

```
def test_prompt_template_renders_correctly():
    from langchain.prompts import PromptTemplate
    from agent.prompts import LOAN_AGENT_SYSTEM_PROMPT

    # The prompt should have required template variables
    prompt = PromptTemplate.from_template(LOAN_AGENT_SYSTEM_PROMPT + """
Tools: {tools}
Tool names: {tool_names}
Input: {input}
Scratchpad: {agent_scratchpad}""")

    rendered = prompt.format(
        tools="tool list",
        tool_names="tool_name_list",
        input="test question",
        agent_scratchpad=""
    )
    assert "test question" in rendered
    assert len(rendered) > 0
```

END-TO-END REASONING TESTS (6 cases)

TC-01-P3-E2E-01: Agent Answers Application Status Query

Category: End-to-End | **Priority:** High **Prerequisites:** Phase 1 API running, application 1 exists

```
def test_agent_answers_status_query(running_api):
    from agent.agent import build_agent, run_agent
    executor = build_agent()

    result = run_agent("What is the status of application 1?", executor)

    output = result.get("output", "").lower()
    assert len(output) > 10
    # Should mention a status
    statuses = ["submitted", "under_review", "approved", "rejected", "disbursed",
"not found"]
    assert any(s in output for s in statuses)
```

TC-01-P3-E2E-02: Agent Uses Multiple Tools for Combined Query

Category: End-to-End | **Priority:** High **Prerequisites:** Phase 1 API running, ChromaDB populated

```
def test_agent_uses_multiple_tools(running_api):
    from agent.agent import build_agent, run_agent
    executor = build_agent()

    result = run_agent(
        "What is the status of application 1 and what documents are needed for a
home loan?",
        executor
    )

    # Should have used at least 2 tools
    steps = result.get("intermediate_steps", [])
    tool_names_used = [step[0].tool for step in steps]

    assert len(tool_names_used) >= 2
    assert "get_application_details" in tool_names_used
    assert "search_loan_policy" in tool_names_used
```

TC-01-P3-E2E-03: Agent Answers Policy-Only Query with RAG Tool

Category: End-to-End | **Priority:** High **Prerequisites:** ChromaDB populated

```
def test_agent_answers_policy_only_query():
    from agent.agent import build_agent, run_agent
    executor = build_agent()

    result = run_agent("What is the minimum income required for a home loan?",
executor)
    steps = result.get("intermediate_steps", [])
    tool_names_used = [step[0].tool for step in steps]

    # Should use policy search, not application APIs for this question
    assert "search_loan_policy" in tool_names_used
    output = result.get("output", "").lower()
    assert any(term in output for term in ["income", "monthly", "annual",
"40,000"])
```

TC-01-P3-E2E-04: Agent Summarizes Multiple Applications

Category: End-to-End | **Priority:** Medium **Prerequisites:** Phase 1 API running, multiple applications exist

```
def test_agent_summarizes_multiple_applications(running_api):
    from agent.agent import build_agent, run_agent
    executor = build_agent()

    result = run_agent("How many applications are currently pending review?",
executor)
    output = result.get("output", "")

    assert len(output) > 0
    # Should mention a number or "no applications"
    assert any(char.isdigit() for char in output) or "no application" in
output.lower()
```

TC-01-P3-E2E-05: Agent Handles Ambiguous Query

Category: End-to-End | **Priority:** Medium

```
def test_agent_handles_ambiguous_query():
    from agent.agent import build_agent, run_agent
    executor = build_agent()

    # Vague query that should produce some response
    result = run_agent("Tell me about loans", executor)
    output = result.get("output", "")

    assert len(output) > 10
    # Should not error out
```

```
assert "error" not in output.lower() or "I" in output # Has a coherent response
```

TC-01-P3-E2E-06: Agent Reasoning Traced in LangSmith

Category: End-to-End | **Priority:** High **Prerequisites:** LangSmith credentials set

```
def test_full_reasoning_chain_traced():
    import os, time
    from langsmith import Client
    from agent.agent import build_agent, run_agent

    executor = build_agent()
    run_agent("What is the dashboard summary?", executor)

    time.sleep(3) # Wait for trace upload

    client = Client(api_key=os.getenv("LANGCHAIN_API_KEY"))
    runs = list(client.list_runs(
        project_name="AI-Readiness-POC-01-P3",
        limit=3
    ))

    assert len(runs) > 0, "No traces in AI-Readiness-POC-01-P3"
```

Running Phase 3 Tests

```
# Start Phase 1 API first
cd backend && uvicorn app.main:app --port 8000 &

# Ensure Phase 2 ChromaDB is populated
python rag/ingest.py

# Run all Phase 3 tests
pytest tests/phase3/ -v

# Run with API dependency skip
pytest tests/phase3/ -v -m "not requires_api"

# Generate report
pytest tests/phase3/ --junitxml=results/phase3-results.xml
```